1. INTRODUCERE

OBIECTIVE

Principalele obiective ale acestui capitol introductiv sunt următoarele:

- caracterizarea noţiunii de informaţie, reprezentarea şi prelucrarea acesteia în sistemele tehnice;
- obţinerea prin rafinări succesive a unui model structural al unui Calculator Numeric;
- prezentarea unui model funcţional al unui Calculator Numeric, ca o ierarhie de maşini virtuale;
- analiza din punct de vedere istoric, a evoluţiei echipamentelor de prelucrare numerică a datelor;
- prezentarea funcţiilor primitive şi unităţilor funcţionale ale Calculatorului Numeric, posibilităţi de realizare a acestora.

CUPRINS:

1.1 INTRODUCERE 1
1.2 CALCULATOR NUMERIC. SISTEM DE PRELUCRARE A INFORMAŢIILOR 3
1.3 MODELUL STRUCTURAL AL UNUI CALCULATOR NUMERIC 5
1.3.1 CN cu sistem de I/E cu transfer programat 9
1.3.2 CN cu sistem de I/E cu transfer prin DMA 9
1.3.3 CN cu sistem de I/E cu transfer prin canal de I/E 10
1.4 MODELUL FUNCŢIONAL AL UNUI CN 11
1.5 UNITĂŢI FUNCŢIONALE ALE UNUI CALCULATOR NUMERIC 15
1.5.1 Memoria 16
1.5.2 Unitatea centrală de prelucrare 19
1.5.3 Unitatea de comandă 23
1.5.4 Setul de instrucţiuni 25
1.5.5 Subsistemul de intrare ieşire 27
1.5.6 Limbaje de programare 27
1.6 ELEMENTE DE ARHITECTURĂ ŞI STRUCTURĂ A UNUI CN 31
1.6.1 Concepte de bază31
1.6.2 Moduri de adresare 33

1.1 Calculator Numeric - Sistem de prelucrare a informaţiilor

Vom considera informaţia o noţiune primară, profundă, similară noţiunii de materie. Informaţia poate fi regăsită în întregul ciclu material, începând cu lumea cuantică şi microscopică şi continuând în lumea vie şi sistemele tehnice. În legătură cu informaţia, vom considera următoarele probleme :

- structurarea şi reprezentarea;
- stocarea pe diferite suporturi de informaţie;
- prelucrarea;
- transmiterea.

În sistemele tehnice, informaţia este reprezentată în principal sub formă analogică şi sub forma discretă (numerică). În mod corespunzător s-au dezvoltat sisteme tehnice de prelucrare a informaţiilor printre care calculatoarele analogice şi calculatoarele numerice au un rol important. Deşi calculatoarele analogice universale prezentau o serie de avantaje privind viteza de lucru, lucrul în timp real, supleţea soluţiilor unor probleme prin analogii, calculatoarele numerice au cunoscut o dezvoltare mult mai rapidă în ultimii ani înlocuind aproape în totalitate pe cele analogice. Datorită inovaţiilor tehnologice şi dezvoltării circuitelor integrate pe scară largă şi foarte largă s-a eliminat treptat handicapul privind viteza de lucru, în schimb precizia de reprezentare şi prelucrare poate fi făcută oricât de mare, stabilitatea şi robusteţea calculatoarelor numerice sunt net inferioare, iar posibilităţile de structurare, reprezentare, şi stocare a informaţiilor sunt nelimitate. Desigur că informaţiile sub formă analogică pot fi în continuare prelucrate după ce au fost în prealabil convertite în formă numerică, rezultatele fiind supuse unei conversii inverse.

Calculatoarele numerice se caracterizează prin faptul că asigură reprezentarea, stocarea, prelucrarea şi transmiterea informaţiilor sub forma discretă, numerică. Astfel, mărimi electrice continue, cum ar fi tensiunea sau curentul electric pot fi memorate prin distingerea diferitelor valori ale acestor mărimi. Cu cât trebuie să se distingă mai multe valori, cu atât separarea dintre două valori adiacente va fi mai mică şi deci memoria va fi mai puţin sigură în funcţionare. Sistemul binar trebuie să distingă doar două valori, deci constituie metoda cea mai sigură de codificare numerică a informaţiilor. Unitatea de informaţie elementară este în acest caz cifra binară denumită "BIT". Un bit poate conţine un "0" sau un "1". Există unele calculatoare despre care se spune că folosesc sistemul zecimal, nu binar. De fapt se utilizează sistemul BCD - Binar Codificat Zecimal care constă în reprezentarea cifrelor zecimale între 0 - 9 prin patru cifre binare, adică patru biţi. Utilizând 16 biţi, pentru reprezentarea numărului 1992 vom obtine:

- zecimal : 0001 1001 1001 0010 şi
- binar: 0111 1100 1000.

Sistemul zecimal se utilizează doar la calculatoare cu aplicaţii strict economice, financiare.

Se observă că 16 biţi pot reprezenta în zecimal numere cuprinse între 0 - 9999, deci 10000 de combinaţii, pe când în binar se pot reprezenta 65536 combinaţii diferite. Se poate spune deci că sistemul binar este mai eficient. Dar să analizăm ce s-ar întâmpla dacă cineva ar inventa un dispozitiv electronic, foarte sigur în funcţionare, capabil să memoreze cifrele de la 0 - 9 prin împărţirea domeniului de la 0 - 10V, în 10 intervale, de exemplu. Patru asemenea dispozitive ar putea memora 10000 de combinaţii, adică valori între 0 - 9999. Utilizând sistemul binar, aceleaţi 4 elemente ar putea memora 16 combinaţii. Este evident că în acest caz sistemul zecimal ar fi mult mai eficient.

O altă problemă apare datorită faptului că informaţia prezintă două aspecte: sintactic şi semantic. Prin aspectul său sintactic, informaţia poate fi înţeleasă sub aspectul ideilor încapsulate, mesajele pe care le transmite, etc.

Calculatoarele numerice în acceptiunea clasică, sunt considerate principalele unelte pentru prelucrarea automată a informaţiilor, ele fiind capabile să trateze informaţiile numai sub aspect sintactic. Fără să cunoască aspectul semantic al informaţiilor pe care le prelucrează, calculatoarele nu pot să ia decizii proprii, ele fiind condamnate să facă doar ceea ce a fost specificat în prealabil, precis şi fără echivoc printr-o descriere care poartă numele de algoritm. Prin algoritm se asigură şi respectarea unui principiu de bază în prelucrarea automată a informaţiilor, concordanţa aspectului sintactic cu cel semantic. Dificultăţile care apar în acest sens i-au determinat pe utilizatorii lui să afirme că, de multe ori, calculatoarele crează mai multe probleme decât rezolvă. Ideea că un calculator nu poate face decât ceea ce i se spune, este deja depăşită. La ora actuală se fac eforturi din ce în ce mai mari pentru a reprezenta şi prelucra cu ajutorul calculatoarelor informaţii cu aspect semantic. Realizarea acestui deziderat în condiţii de eficienţă acceptabile permite utilizarea calculatorului într-o manieră cu totul nouă. El este capabil să reprezinte şi să prelucreze cunoştinţe, să înveţe din experienţa anterioară, să îmbunătăţească sau să dezvolte noi metode de rezolvare a unor probleme.

Oricum, multe probleme care implică prelucrarea unui volum mare de date, eventual cu limitări stricte de timp, pot să fie rezolvate numai cu ajutorul calculatorului. În mod "clasic", aplicaţiile calculatorului pot să fie grupate în aplicaţii cu caracter ştiinţific care necesită precizie şi viteză mare de efectuare a calculelor matematice si aplicaţii cu caracter de gestiune care necesită păstrarea si regăsirea rapidă a informaţiilor. Pe de altă parte, aplicaţii de tipul: prevederea timpului probabil sau proiectarea unui avion necesită atât numeroase calcule stiinţifice, cât şi manipularea unui volum foarte mare de informaţii. să considerăm de exemplu câteva tipuri de aplicaţii specifice calculatoarelor:

- **Calcule ştiintifice:** În experimentele stiinţifice şi de laborator calculatoarele pot să fie utilizate pentru culegerea, validarea şi prelucrarea datelor preluate de la

diferiţi senzori sau aparate. Domenii tipice: telemetrie, radiolocaţie, spectrografie, etc.

- **Proiectare asistată:** Calculatorul este deosebit de util în multe domenii de proiectare ca de exemplu: proiectarea măştilor pentru VLSI, proiectarea circuitelor imprimate, proiectarea fuzelajelor şi aripilor de avion, a caroseriilor de automobil, etc.
- **Conducerea proceselor**: Calculatorul este o unealtă foarte utilă în fabricarea şi controlul automat al produselor, de exemplu: comanda maşinilor unelte, controlul proceselor de difuzie şi încapsulare într-o fabrică de circuite integrate, conducerea roboţilor din celulele flexibile de fabricaţie, etc.
- **Simulare**: Datorită utilizării calculatoarelor a apărut între cercetarea fundamentală şi cea aplicativă un nou nivel de cercetare, prin simulare. Acest tip de abordare este foarte util pentru situaţiile în care în condiţii reale experimentele pot să fie prea costisitoare, periculoase sau chiar imposibil de realizat. să considerăm de exemplu cazul în care se doreşte studiul efectului erupţiei unui vulcan. Evident, vulcanul nu poate să fie convins să erupă, dar se poate realiza o simulare în care să se ţină seama de cât mai mulţi factori care caracterizează fenomenul natural.
- **Instruire asistată**: Calculatorul poate să constituie un instrument foarte util în creşterea eficienţei procesului de predare - învăţare - verificare, pentru stimularea gândirii creative, logice.

1.2 Modelul structural al unui Calculator Numeric

Atât funcţional, cât şi prin construcţie calculatorul este un sistem logic. O caracteristică esenţială a unui astfel de sistem este posibilitatea de a descrie funcţionarea acestuia independent de detaliile de realizare fizică concrete. Astfel, indiferent de domeniul de aplicaţii, calculatorul poate să fie privit într-o primă aproximaţie ca o cutie neagră având o comportare deterministă, predictibilă, primind la intrare date pe care le prelucrează si generând la ieşire rezultate.

Sistemul din Fig. 1.1 poate să fie interpretat ca realizând o aplicaţie (transformare) din domeniul de intrare în domeniul de ieşire, comportarea sa fiind materializată în relaţiile dintre intrări, ieşiri şi timp.

Prin rafinări succesive ale acestui model foarte general se pot obţine modele funcţionale sau constructive cu diferite grade de detaliere. Sistemul are un set de terminale de intrare "I" şi un set de terminale de ieşire "E".

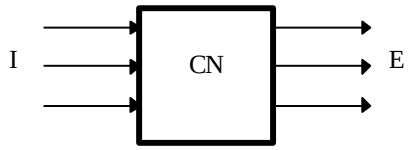


Fig. 1.1 Modelul general al unui calculator numeric

Fiecare terminal poate să fie considerat în una din cele două stări: activ sau inactiv, respectiv ca având valoarea 1 sau 0. O combinaţie de unităţi şi zerouri formează un vector binar. Cutia neagră poate fi instruită ce să facă arătând care este vectorul de ieşire pentru un vector de intrare fixat. După ce se repetă acest experiment pentru mai mulţi vectori I/E (perechi de vectori de intrare - ieşire) ar fi de dorit ca automat cutia neagră să arate vectorul corect la ieşire când se fixează un vector dat la intrare. Ideal ar fi ca dacă există o regulă clară de corespondenţă între vectorii de intrare şi cei de ieşire, C.N. să extragă această regulă şi s-o păstreze în conexiunile interne astfel încât să genereze vectorul de ieşire corect pentru un vector pe care nu l-a mai avut la intrare înainte. În acest model C.N. apare ca un sistem definit prin mulţimea perechilor de intrări-ieşiri care se află permanent într-o anumită stare, definită ca un marcaj (sau conexiuni) ataşat unei submulţimi de perechi de intrări-ieşiri care satisfac anumite condiţii de consistenţă. Această submultime va fi considerată un subsistem.

În general nu este practic să se enumere toate perechile de I/E care definesc un astfel de sistem preferându-se caracterizarea acestuia printr-o relaţie de intrare-ieşire, adică printr-un algoritm sau o ecuaţie care se poate utiliza pentru generarea tuturor perechilor de I/E care-i aparţin.

Pe de altă parte, un astfel de sistem nu implică întotdeauna un vector unic de ieşire precum un vector de intrare. Dimpotrivă, unui vector de intrare îi va corespunde un număr de vectori de ieşire posibili, fiecare conţinând un răspuns posibil al vectorului de intrare dat. Această neunicitate a răspunsului la o intrare dată reflectă dependenţa ieşirii atât de intrare cât şi de starea initială (sau starea precedentă). Această neunicitate a dependenţei ieşirii de intrare arată că un astfel de sistem este mai degrabă o relaţie decât o funcţie sau un operator.

Reluând modelul din Fig. 1.1, se poate constata că relaţia dintre vectorii de intrare şi ieşire este prea complicată pentru a fi încapsulată într-un sistem cu conexiuni directe între intrări şi ieşiri. Astfel este necesară introducerea unor nivele intermediare în cutia neagră formate din unităţi funcţionale capabile să extragă diferite caracteristici ale vectorilor de intrare, care eventual, să genereze ieşirea corectă. O primă observaţie trebuie făcută în legătură cu mulţimile de intrări şi ieşiri. Acestea nu sunt întotdeauna sub formă de vectori formaţi din 1 şi 0 şi nu au structura adecvată unui anumit tip de prelucrare. Este astfel necesară prevederea unui subsistem de intrare care asigură preluarea datelor din mediul extern şi aducerea lor la o formă standard de intrare în vederea prelucrării. După prelucrare, datele trebuie transmise spre mediul extern în forma cerută, ceea ce implică necesitatea unui

subsistem de ieşire. Se obţine, în acest mod, un model structural al unui C.N., prezentat în Fig. 1.2.

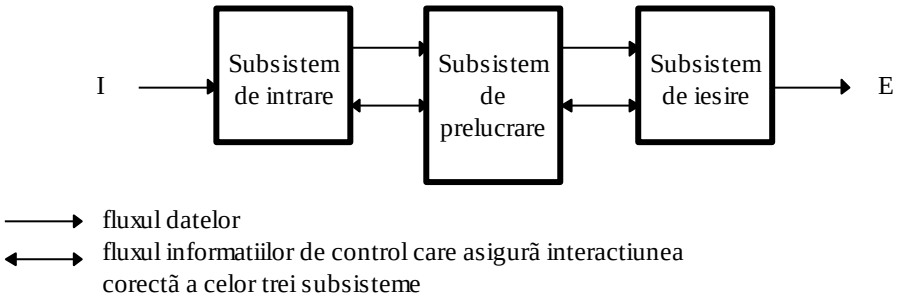


Fig. 1.2 Modelul structural al unui C.N.

Subsistemul de intrare trebuie să asigure atât conversia în semnale electrice a datelor reprezentate pe diferite suporturi în mediul extern cât şi transformarea acestor semnale în vectori binari cu structura cerută de subsistemul de prelucrare. În mod similar subsistemul de ieşire trebuie să asigure operaţii similare, în ordine internă, pentru rezultatele trimise spre mediul extern.

Subsistemul de prelucrare trebuie să asigure reprezentarea internă, stocarea şi prelucrarea datelor şi schimbul de informaţii cu subsistemul de I/E. Se obţine astfel structura unui C.N. cum este reprezentată în Fig. 1.3.

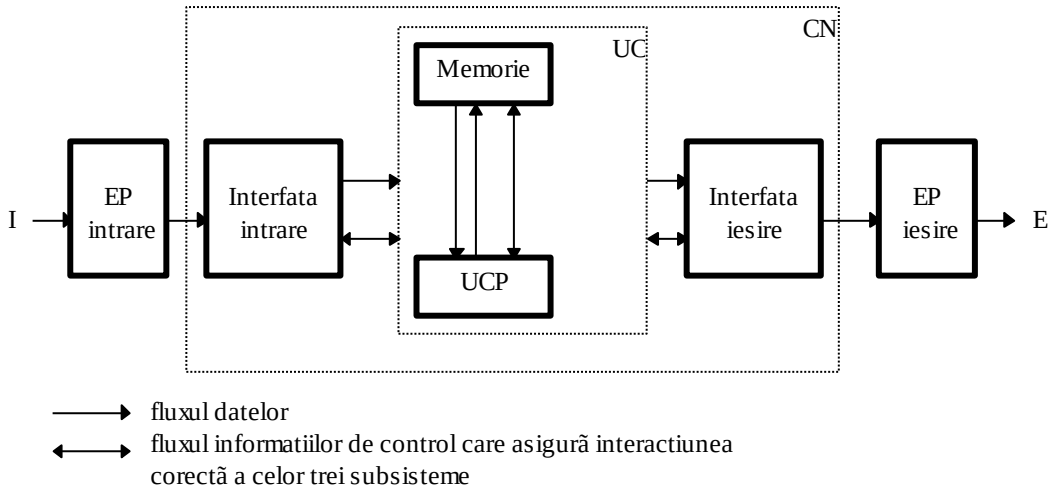
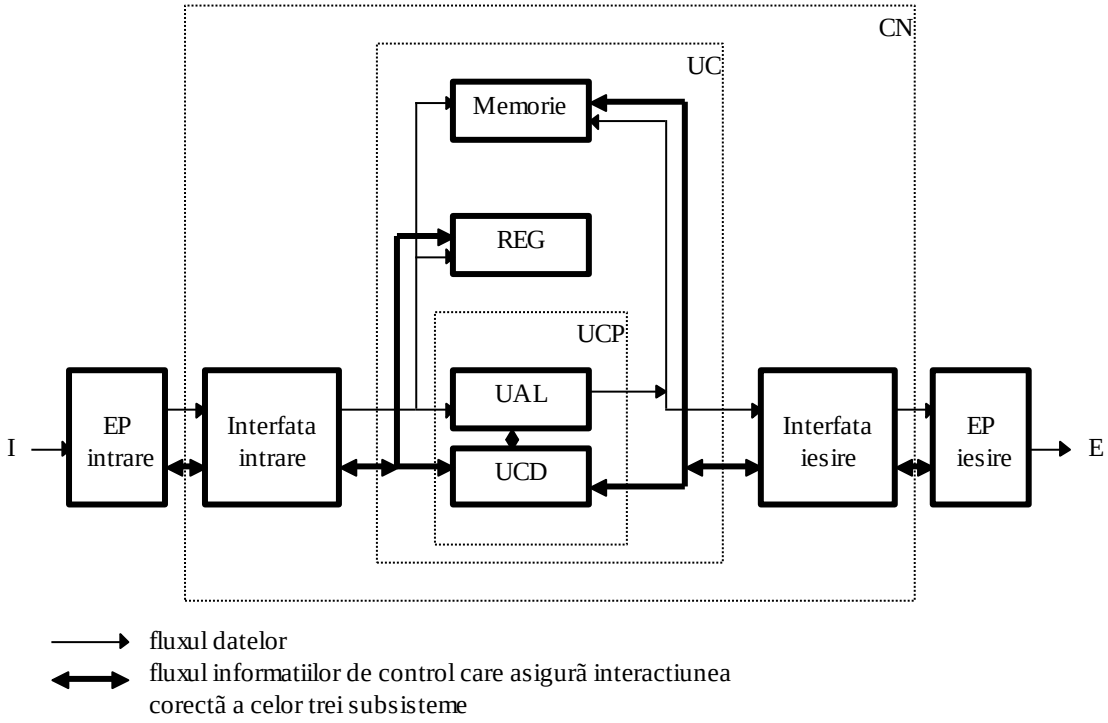


Fig. 1.3 Structura unui C.N.

Subsistemul de prelucrare, împreună cu interfaţa de I/E formează calculatorul propriuzis. Subsistemul de prelucrare constituie nucleul central al C.N., motiv pentru care, va fi denumit UNITATEA CENTRALA (UC). UC este formată dintr-un subsistem de memorie internă şi o Unitate Centrală de Prelucrare. Din considerente de eficienţă şi cost, memoria este organizată în general, pe două nivele. O memorie de capacitate mică şi viteză mare, asociată direct UCP

sau chiar făcând parte din aceasta, formată din registre rapide şi o memorie mai lentă, de capacitate mare. UCP are atât rolul de a asigura funcţiile pentru prelucrări aritmetice şi logice de bază cât şi coordonarea transferului informaţiilor între elementele componente. Se obţine astfel în Fig. 1.4, schema bloc a unui sistem de calcul care include CN propriu-zis, echipamentele periferice de I/E şi sistemul de programe care asigură o utilizare eficientă a întregului sistem.



UC + INTEF. de INTRARE şi de IESIRE = C.N.

CN + EP + SOFTWARE = SISTEM DE CALCUL

Fig. 1.4 Schema bloc a unui SISTEM DE CALCUL

Conexiunile efective între diferite elemente funcţionale ale UCP, între UCP şi memorie, între UC şi interfeţele de I/E se pot face în diferite moduri, obţinându-se în mod corespunzător diferite structuri de C.N.

Astfel dacă se consideră fluxul informaţiilor între UC şi interfeţele de I/E se obţin trei tipuri de transferuri de I/E :

- Subsistem de I/E cu transfer programat;
- Subsistem de I/E cu transfer prin acces direct la memorie;
- Subsistem de I/E cu transfer prin canal de I/E.

1.2.1 CN cu sistem de I/E cu transfer programat

Schema bloc este arătată în Fig. 1.5

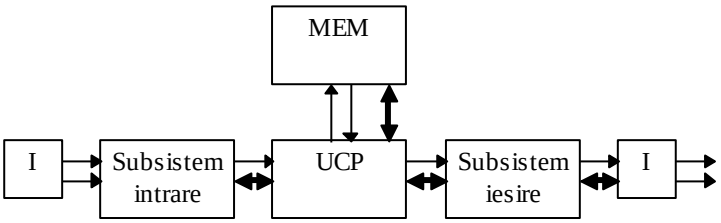


Fig. 1.5. CN cu transfer programat

În această schemă de organizare datele de I/E circulă direct prin registrele UCP sub controlul UC din UCP. Transferul se efectuează cuvânt cu cuvânt şi pentru fiecare cuvânt transferat UCP execută o secvenţă de instrucţiuni, numită program de I/E. Soluţia este în general simplu de implementat şi ca resurse hardware şi ca efort de programare dar se pretează numai la echipamente cu volum de date şi rată de transfer scăzute.

1.2.2 CN cu sistem de I/E cu transfer prin DMA

În cazul unor echipamente periferice cu viteza de transfer ridicată, discuri şi benzi magnetice de exemplu, programul care se execută pentru fiecare cuvânt transferat poate să fie prea lung, ceea ce ar duce la pierderea unor cuvinte. Pe de altă parte, volumul de date fiind ridicat, UCP ar fi ocupată mult numai cu transferul datelor de I/E. În astfel de situaţii se utilizează transferul prin acces direct la memorie. Fluxul informaţiilor în acest caz este arătat în Fig. 1.6.

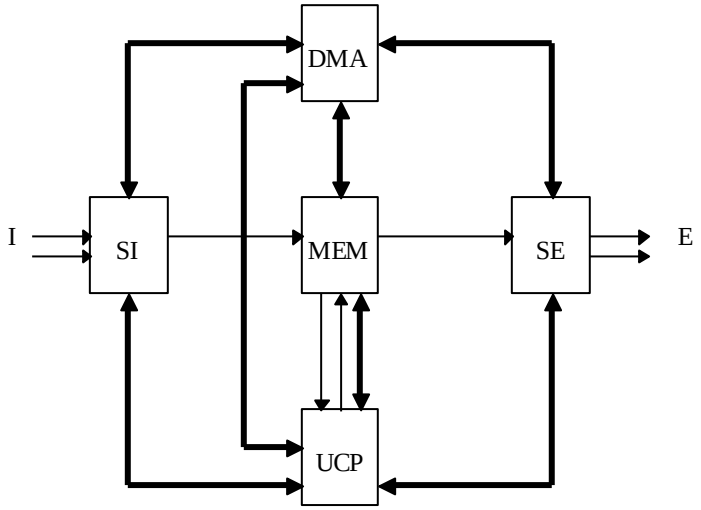


Fig. 1.6 CN cu transfer prin DMA

După iniţierea unui transfer, UCP poate continua cu alte activităţi. Transferul întregului bloc se execută sub controlul modului DMA. La sfârşitul transferului sau la apariţia unor evenimente, de exemplu erori de transfer, se cere din nou intervenţia UCP.

1.2.3 CN cu sistem de I/E cu transfer prin canal de I/E

Un grad şi mai mare de independenţă dintre UCP şi subsistemul de I/E se obţine prin prevederea unui procesor specializat, numit canal de I/E, capabil să execute transferul prin acces direct la memorie, a unei înlănţuiri de blocuri şi să rezolve unele situaţii de eroare apărute în cadrul transferului, fără intervenţia UCP. Schema bloc este arătată în Fig. 1.7.

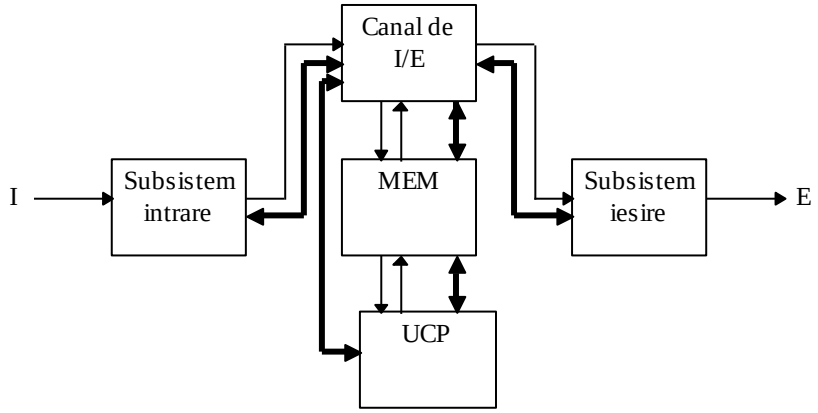


Fig. 1.7 CN cu transfer prin canal de I/E

Procesorul de canal execută programe de canal pregătite în memorie de către UCP. Programele de canal specifică toţi parametrii transferului şi pot exista mai multe programe de canal înlănţuite. Productivitatea sistemului este mult mai mare în acest caz.

1.3 Modelul funcţional al unui CN

În paragraful precedent s-a introdus modelul structural al unui CN prin rafinarea succesivă a unui model foarte general de sistem dinamic specificat ca o relaţie între mulţimea intrărilor şi ieşirilor. În final s-a arătat că sistemul de calcul este format din resursele fizice care alcătuiesc partea hardware şi un sistem de programe de bază şi aplicaţii care alcătuiesc partea de software a sistemului de calcul. Există şi un nivel intermediar între hardware şi software, denumit firmware, care există din software incorporat în hardware în momentul fabricaţiei calculatorului. Programele care controlează direct resursele hardware, care nu se schimbă atâta timp cât structura hardware nu se schimbă şi programele care trebuie să fie prezente în calculator imediat după alimentarea cu energie a calculatorului, constituie partea de firmware.

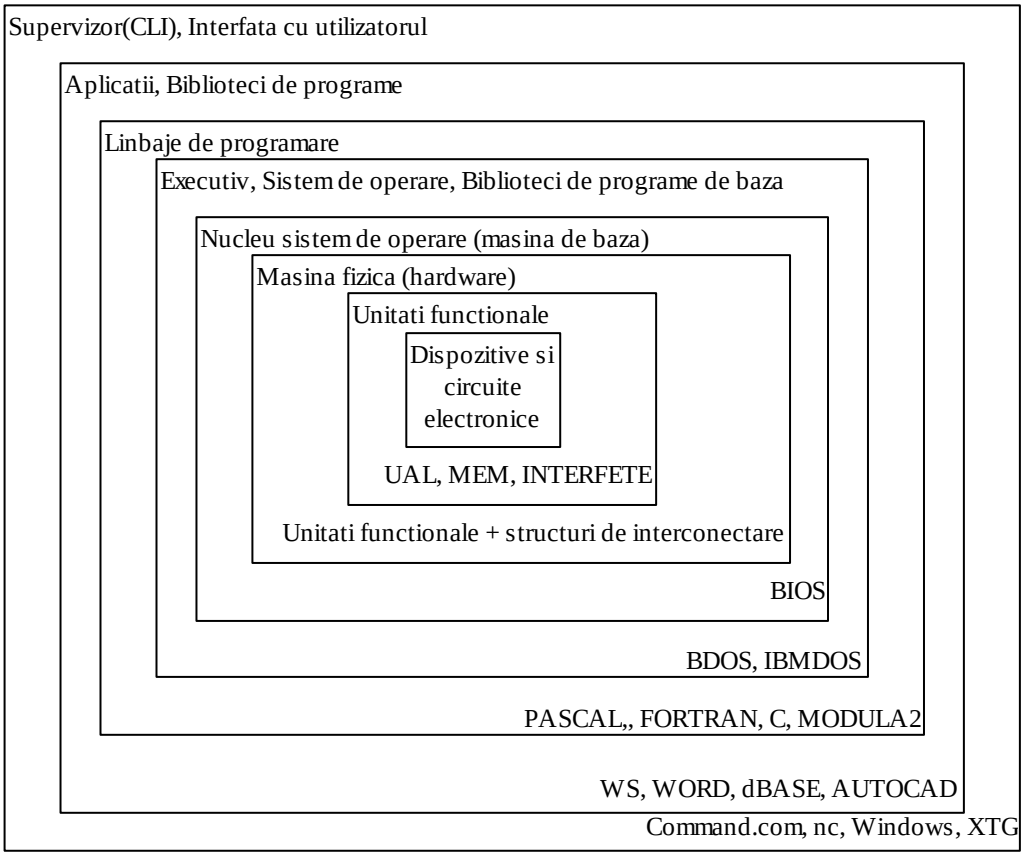


Fig.1.8. Modelul funcţional al unui CN

De fapt partea de hardware şi cea de software sunt echivalente. Orice operaţie efectuată prin software poate fi implementată direct în hardware şi orice instrucţiune efectuată prin hardware, poate fi simulată prin software. Decizia de a implementa anumite funcţii în hardware şi altele în software este luată având în vedere anumiţi factori cum ar fi: costul, viteza de prelucrare, raportul performanţe pe cost, siguranţa în funcţionare şi frecvenţa unor modificări posibile. La primele calculatoare această decizie era simplu de luat. În hardware erau implementate câteva funcţii foarte simple de tipul: adună două numere întregi, salt la o altă instrucţiune. Toate celelalte operaţii erau realizate explicit prin software, aceasta din cauza constrîngerilor tehnologice existente. Pe măsura evoluţiei tehnologiei are loc o tendinţă inversă.

Proiectanţii de hardware au selectat din ce în ce mai multe funcţii ce se executau suficient de frecvent motivând implementarea lor în hardware pentru a fi executate mult mai repede. Astfel la ora actuală există circuite specializate care efectuează calcule cu numere reale cu precizie foarte mare, procesoare grafice care printr-o simplă comandă execută o transformare a unei figuri complexe etc. Oricât de complexe ar fi funcţiile realizate în hardware, utilizatorul va dori să promoveze operaţii şi mai complexe prin software. Astfel din punctul de vedere al utilizatorului sistemul de calcul este privit ca o ierarhie de uniuni virtuale fiecare oferindu-i facilităţi diferite de programare. Modelul funcţional prezentat în Fig.1.8 reflectă atât organizarea sistemului de calcul cât şi funcţionarea acestuia privind fluxul informaţiilor plecând de la programul utilizatorului pînă la obţinerea informaţiilor dorite.

În literatura de specialitate există modele similare cu acesta, altele sunt destul de diferite. În acest model s-a urmărit pe de o parte aspectul funcţional, nivelele ierarhice sunt astfel alese încât să reflecte cât mai bine fluxul datelor şi acţiunilor în sistemele moderne de calcul iar pe de altă parte aspectul didactic, fiecare nivel ierarhic este obiectul unor cercuri clar delimitate, intercalate prin conţinut şi planificare în timp.

NIVELUL 1. - Dispozitivele şi circuitele electronice reflectă atât gradul de integrare tehnologică cât clasa de performanţe în care se încadrează calculatorul. Astfel, un calculator bazat pe circuite ECL (circuite logice cu cuplaj prin emitor) va avea o viteză de prelucrare ridicată, destinat unor aplicaţii complexe, cu multe calcule. Un calculator bazat pe microprocesoare 80386 şi 80387 şi memorii de 1-2 biţi pe pastilă va fi un calculator performant realizat într-o tehnologie avansată.

NIVELUL 2. - Unităţile funcţionale reflectă într-o bună măsură modularitatea sistemului, poziţionarea şi implementarea funcţiilor primitive în unităţi funcţionale avînd în vedere criterii funcţionale, constructive, şi aspecte privind testabilitatea. La acest nivel încă nu este posibilă programarea calculatorului. Este posibilă însă microprogramarea. Diferite unităţi funcţionale conţin programe implementate sub formă de firmware. Astfel unitatea centrală de prelucrare poate conţine programe firmware pentru controlul componentelor din care este alcătuită,

programe pentru menţinerea configuraţiei de bază, pentru cazul de timp real al calculatorului, programe de depanare, etc. Un canal de I/E pentru disc include programe de testare a subsistemului de discuri magnetice, programele de I/E de nivel scăzut, programarea circuitelor integrate pe scară largă care intră în componenţa sa.

NIVELUL 3. - Maşina fizică reprezintă, împreună cu echipamentele de I/E, partea de hardware propriu-zisă a calculatorului. Acest nivel reflectă structura şi organizarea internă a calculatorului, modul de interconectare şi interacţiunile dintre unităţile funcţionale, fluxul datelor de I/E. De asemenea acest nivel reflectă şi arhitectura calculatorului aşa cum este el văzut de un programator la nivelul funcţiilor primitive .

NIVELUL 4. - Maşina de bază reprezintă calculatorul propriu-zis. Nucleul sistemului de operare trebuie văzut mai mult ca firmware decât ca software şi este format dintr-o colecţie de programe orientate pe maşina fizică şi care oferă programatorului de sistem o maşină de bază mai evoluată, uniformă şi stabilă în timp, primind modelul de acces la funcţiile primitive. Astfel modificările din nivelele 1, 2 şi 3, vor fi preluate de acest nivel fără a fi simţite de nivelele ierarhice superioare. Acest nivel asigură în principal funcţiile de I/E de bază (BIOS la PC) şi funcţii de acces direct la resursele fizice pentru testarea şi punerea la punct a programelor.

NIVELUL 5. - Executivul sistemului de operare reprezintă o colecţie de programe de bază care asigură pe de o parte o maşină cât mai eficientă şi comodă de utilizat pentru programator şi o utilizare cât mai eficientă a resurselor hardware şi software, pe de altă parte. Principalele funcţii ale unui sistem de operare sunt:

- controlul încărcării şi execuţiei programelor, care asigură încărcarea în memoria internă a programelor preluate din memoria externă, lansarea în execuţie şi supravegherea execuţiei acestora;
- operaţii de I/E mai evoluate, bazate pe funcţiile oferite de maşina de bază;
- gestiunea fiţierelor, care asigură crearea întreşinerea şi utilizarea fiţierelor de date pe suporturile externe (magnetice, optice, etc);
- gestiunea memoriei care asigură utilizarea şi protecţia memoriei interne în condiţiile în care mai multe procese pot fi active la un moment dat în memorie;
- facilităţi de dezvoltare a programelor privind programarea modulară, deformarea;
- crearea, întreţinerea şi utilizarea bibliotecilor de programe de bază şi aplicaţii.

NIVELUL 6. Limbaje de programare. La acest nivel am situat limbajele de programare indiferent că sunt de nivel ridicat sau scăzut deoarece prin intermediul acestora programatorul are la dispoziţie un mod esenţial de comunicare cu calculatorul. Fiecare limbaj are un vocabular de aproximativ 100 de cuvinte şi simboluri şi este bazat pe o gramatică strict definită, fără excepţii. Propoziţiile dintr-un limbaj pot fi uşor traduse în propoziţii dintr-un alt limbaj. Operaţia de conversie a unui program dintr-un limbaj în altul se numeşte translatare. Programul original se numeşte program sursă şi este scris în limbajul sursă. Atât limbajul sursă cât limbajul destinaţie, în care este translatat programul, definesc nivelele maşinii virtuale într-o schemă similară celei din Fig.1.8 dar mai detaliată. Dacă ar exista un procesor care să interpreteze direct limbajul sursă nu ar mai fi nevoie de translatare. În prezent există peste 100 de limbaje de programare clasificate după diferite criterii. Astfel sunt:

- Limbaje orientate maşină sau limbaje de nivel scăzut, care necesită din partea programatorului o cunoaştere bună a arhitecturii calculatorului. Limbajul oferă facilităţi de acces direct la aceste resurse, permiţând dezvoltarea unor programe foarte eficiente din punct de vedere al utilizării resurselor pe seama unui efort mai mare din partea programatorului. Limbajele orientate maşină pot la rândul lor să fie clasificate în:
  - limbaje maşină sau limbaje obiect care implică specificarea direct în binar, octet, sau hexazecimal a instrucţiunilor ce se vor executa direct de către maşina fizică. Programarea în limbaj maşină este rareori utilizată eventual pentru câteva instrucţiuni, în procesul de depanare a programelor;
  - limbaje de asamblare care pot fi privite ca o reprezentare simbolică pentru limbajele maşină. Instrucţiunile din programul sursă în limbaj de asamblare, au corespondent direct în limbaj maşină. Utilizarea unor nume simbolice pentru instrucţiuni şi posibilitatea definirii unor adrese simbolice uşurează mult programarea faţă de programarea în limbaj maşină. Translatarea programelor din limbaj de asamblare în limbaj maşină se face de către asamblor.
- limbaje orientate pe probleme sau limbaje de nivel înalt în care utilizatorul îşi scrie programele într-o manieră mai apropiată de modul de comunicare uzual. Detaliile interne ale arhitecturii şi structurii maşinii de bază sunt mult mai puţin vizibile decât în corpul limbajelor de asamblare. Limbajele de nivel înalt sunt mai eficiente pentru programator dar, în general mai ineficiente pentru maşină. Programele scrise în limbajele de nivel înalt în general sunt compilate, translatate de către compilatoare, direct în limbaje obiect pentru a fi excutate de către maşina de bază. La rândul lor limbajele de nivel înalt pot fi clasificate după diferite criterii, astfel:

- limbaje orientate pe calcule ştiintifice: C, C++, JAVA, FORTRAN, APL, BASIC, ….;
- limbaje pentru calcule economice: COBOL, SNOBOL….

Un alt model de clasificare ar fi după modul în care programatorul verifică acţiunile ce trebuie efectuate în cursul executării programului. Astfel sunt:

- limbaje imperative, în care programatorul verifică, pas cu pas, ce trebuie făcut pentru rezolvarea problemei respective: FORTRAN, PASCAL, C, MODULA, etc;
- limbaje declarative în care programatorul face doar o descriere, într-o formă dată a soluţiei la problemă, de exemplu: LISP, PROLOG, SMALLTALK.

Efectuarea unor studii statistice a evidenţiat faptul că, uzual, un procent de 1% dintr-un program este responsabil de 50% din timpul de execuţie al progamului, iar un procent de 10% din program este responsabil de 90% din timpul de execuţie. Aceasta sugerează ideea utilizării unor soluţii mixte de dezvoltare a programelor. Mai întâi programul este scris total în limbaj de nivel înalt, apoi este evaluat şi măsurat şi anumite porţiuni rescrise în limbaj de asamblare. Unele compilatoare chiar translatează programul sursă în limbaj de asamblare pentru a fi optimizat şi apoi translatat în cod maşină.

NIVELUL 7 - nivelul aplicaţiilor reprezintă nivelul cel mai important pentru utilizator. Bibliotecile de programe de la acest nivel reflectă modul în care poate să utilizeze direct calculatorul în diferite aplicaţii şi uşurinţa cu care pot fi create noi programe de aplicaţii în domenii conexe sau total noi.

NIVELUL 8 - nivelul interpretorului limbajului de comandă reprezintă interfaţa şi modul de interacţiune dintre utilizatori şi sistemul de calcul. Selectarea unei acţiuni poate fi greoaie, introducând o serie de texte de la consola operatorului sau din contră, foarte eficientă prin simpla indicare a unei opţiuni afişate pe ecranul consolei. Se observă din această analiză a modelului din Fig. 1.8 că un programator de un anumit nivel nu necesită o cunoaştere detaliată a nivelelor inferioare doar dacă pentru creşterea eficienţei se doreşte elaborarea directă a unor funcţii sau controlul direct al unor resurse de pe nivele inferioare.

1.4 Unităţi funcţionale ale unui Calculator Numeric

În paragrafele anterioare s-au considerat un model structural şi unul funcţional al unui calculator numeric din punctul de vedere al unui utilizator. Aşa cum am mai afirmat, din punct de vedere logic partea de hardware şi cea de software ale unui calculator numeric sunt

echivalente, în sensul că orice funcţie implementată în software poate să fie implementată în hardware şi orice instrucţiune hardware poate să fie simulată prin software. De exemplu, dacă pentru structurile cele mai simple de calculatoare, singura instrucţiune aritmetică disponibilă era instrucţiunea de adunare pentru numere întregi fără semn, celelate operaţii aritmetice realizându-se prin programare pe baza operaţiei de adunare, în prezent coprocesoarele aritmetice sunt capabile să realizeze chiar şi calculul unor funcţii transcendentale. Din punct de vedere metodologic este însă de preferat să analizăm structura hardware separat de sistemul de programe de bază şi aplicaţii.
În continuare vor fi trecute în revistă unităţile funcţionale de bază ale unui calculator numeric.

1.4.1 Memoria

Pentru realizarea funcţiei de memorare se utilizează o mare varietate de dispozitive şi echipamente mergând de la dispozitive rapide, de mică capacitate şi cu un cost ridicat, până la echipamente lente, de mare capacitate şi cu un cost scăzut. Un calculator poate utiliza o ierarhie de astfel de dispozitive, având rapoarte foarte diferite între performanţe.

În general se pot identifica într-un sistem de calcul trei tipuri de memorii:

- Memorii tampon foarte rapide formate din registrele interne ale UCP şi eventual din memorii "cache" de asemenea conţinute în UCP utilizate pentru a memora instrucţiunile sau datele care urmează să fie utilizate imediat. Memorarea informaţiilor în acest tip de memorii se realizează în avans. Aceste memorii sunt de mică capacitate (8 - 16 000 octeţi) dar au un timp de acces foarte mic (zeci de ns).
- Memoria principală a calculatorului este în legătură directă cu UCP şi este o memorie cu acces rapid având în general o capacitate de ordinul 64K octeţi - zeci de M octeţi.
- Memoria auxiliară (externă) este realizată pe baza unor echipamente periferice de tip disc sau bandă magnetică, cu capacitate mare de memorare (360K octeţi - sute M octeţi). Se consideră că memoria auxiliară face parte din subsistemul de intrare / ieşire.

În continuare prin termenul de memorie ne vom referi la memoria principală.

Memoria conţine atât instrucţiuni cât şi date şi este formată, la nivelul cel mai de jos, din elemente cu două stări stabile, deci elemente care pot să memoreze un bit de informaţie. Organizarea biţilor în memoria principală se face sub forma unor vectori formaţi din unităţi elementare de memorie fiecare conţinând un număr fix de biţi. Operaţiile care se execută în legatură cu memoria sunt operaţii de citire şi scriere. La execuţia unei astfel de operaţii UCP

are acces la conţinutul unei locaţii de memorie (cea mai mică unitate de memorie la care se poate face acces). Orice memorie are următoarele caracteristici principale:

- Fiecare locaţie de memorie are acelaşi număr de biţi;
- Fiecare locaţie de memorie are asociat un număr care reprezintă adresa prin care locaţia respectivă poate să fie referită în mod unic;
- Spaţiul de adrese este omogen şi toate locaţiile de memorie sunt echivalente din punctul de vedere al accesului.

Rezultă deci că o locaţie de memorie este caracterizată de două elemente: adresa care reprezinta poziţia relativă în cadrul memoriei (informaţie fixă) şi conţinutul, care este dat de valoarea numerică memorată în locaţia respectivă la un moment dat.

Conţinutul unei locaţii de memorie reprezintă un cuvânt de memorie, numărul de biţi din cuvânt reprezentând lungimea cuvântului.

Pentru fiecare calculator lungimea cuvântului este stabilită corespunzător tipului de aplicaţii în care calculatorul din care face parte prin proiectare urmează să fie utilizat. De exemplu primul microprocesor 4004 care a fost proiectat pentru a realiza prelucrări în BCD (codul de reprezentare a numerelor Binar Codificat Zecimal) a fost proiectat pentru a lucra cu memorii având lungimea cuvântului de 4 biţi. Succesorul său, 8008 care a fost proiectat pentru a lucra cu caractere reprezentate conform standardului ASCII (cod de 8 biţi) a fost proiectat pentru a lucra cu memorii având lungimea cuvântului de 8 biţi. Deoarece în prezent majoritatea calculatoarelor au lungimea cuvântului un multiplu de 8, se obisnuieşte specificarea acestei lungimi în număr de octeţi (un octet = 8 biţi).

Capacitatea memoriei unui calculator este dată de numărul de octeţi sau de cuvinte din care este formată aceasta. De obicei această capacitate se exprimă în K octeţi (1K = 1024) sau în M octeţi (1M = 1024 K) şi mai nou în gigaocteţi (1G = 1024 M). Capacitatea maximă a memoriei adresate la un moment dat de către un calculator este dată de numărul maxim de adrese distincte care pot să fie generate de unitatea centrală de prelucrare. De exemplu dacă un calculator poate să genereze adrese utilizând 16 biţi atunci el va putea să adreseze 2^{16} adrese distincte. Structura bloc a unui subsistem de memorie este prezentată în Fig. 1.9.

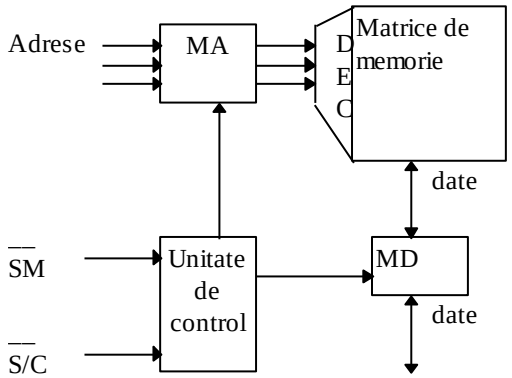
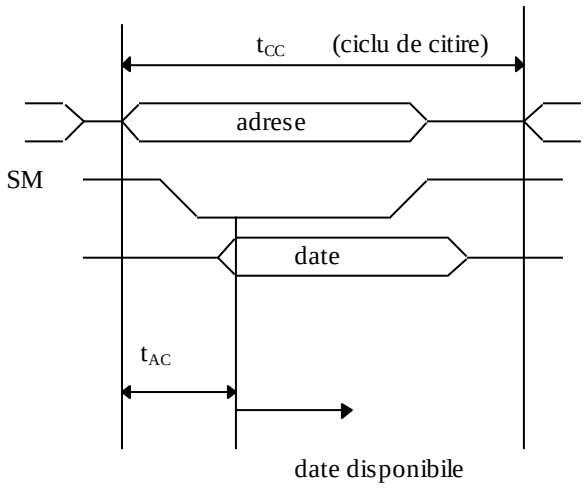
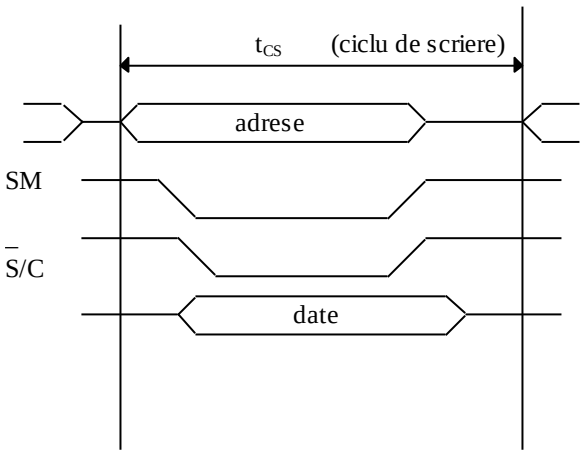


Fig. 1.9 Structura bloc a unui subsistem de memorie



a. diagrama de timp pentru citire



b. diagrama de timp pentru scriere

Fig. 1.10. Diagrame de timp

Registrele de adrese (AM) şi de date (DM) pot să fie conţinute în subsistemul de memorie sau fac parte din alte subsisteme care fac acces la memorie.

Registrul AM conţine adresa de memorie implicată în accesul la memorie. Registrul DM conţine datele care trebuie să fie înscrise în memorie, respectiv datele care se citesc din memorie.

Semnalul SM este un semnal de selecţie a subsistemului de memorie (într-un calculator pot să existe mai multe astfel de subsisteme, eventual fiecare având spaţiul propriu de adrese) iar S/C este un semnal care comandă tipul operaţiei (de scriere sau de citire).

Descrierea funcţionării subsistemului de memorie este prezentată în diagramele de timp din Fig. 1.10.

În figură, cu t_{ac} s-a notat timpul de acces citire ca fiind durata dintre momentul activării adreselor şi momentul în care datele sunt disponibile. Cu t_{cc} s-a notat durata ciclului de citire ca fiind intervalul de timp dintre momentul în care adresele sunt activate pentru citire şi momentul de timp la care se poate face o nouă activare a adreselor. Cu t_{cs} s-a notat durata ciclului de scriere ca fiind intervalul de timp dintre momentul în care adresele sunt activate pentru scriere şi momentul de timp la care se poate face o nouă activare a adreselor.

Se observă că succesiunea de evenimente corespunzătoare efectuării unei operaţii de citire sau de scriere este: stabilirea adresei implicate, generarea semnalului de selecţie, stabilirea semnalului S/C, localizarea adresei în memorie, realizarea transferului de date după care urmează un interval de timp corespunzător refacerii stării circuitelor integrate de memorie pentru a fi capabile să execute un nou acces. Se observă că ciclul memoriei (intervalul de timp dintre două accese la memorie) este format din două intervale de timp: timpul de acces efectiv şi timpul de refacere.

1.4.2 Unitatea centrală de prelucrare

UCP reprezintă nucleul calculatorului. Realizează execuţia instrucţiunilor aduse din memorie asupra unor date aduse din memorie sau obţinute de la subsistemul de intrare. Prelucrările efectuate pot să fie de tip aritmetic sau logic, rezultatele obţinute pot să fie transmise la memorie sau subsistemului de ieşire.

UCP conţine registre care formează o memorie foarte rapidă. Aceste registre pot să conţină date sau componente ale adreselor de memorie. Există calculatoare pentru care unele registre au funcţii dedicate, adică anumite operaţii se pot executa numai asupra valorilor conţinute în anumite registre, în timp ce la alte calculatoare orice registru poate să fie utilizat

pentru orice fel de operaţie. De asemenea UCP conţine un registru, denumit de obicei, registru stare program care memorează informaţii referitoare la starea UCP, la rezultatul execuţiei ultimei instrucţiuni aritmetice sau logice, condiţii de eroare, condiţii care descriu modul în care urmează să se execute următoarele instrucţiuni, etc.

Execuţia operaţiilor aritmetice şi logice se realizează în cadrul Unitaţii Aritmetice şi Logice (UAL). Elementul da bază al unei UAL este un sumator paralel. Viteza de operare a UAL este practic dată de performanţele acestuia. În afară de sumator, UAL conţine şi circuitele combinaţionale care permit selecţia operaţiei aritmetice sau logice executate de către UAL.

În cazul cel mai general o operaţie aritmetică sau logică are doi operanzi şi produce un rezultat. Schema bloc a unei UAL este prezentată în Fig. 1.11.

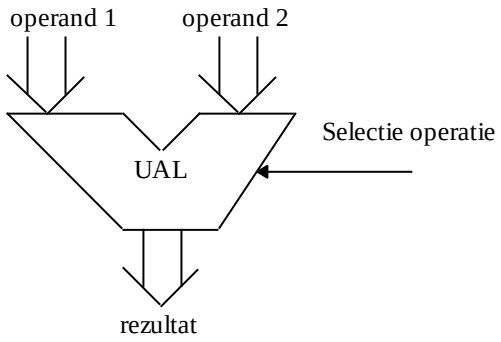


Fig. 1.11 Schema bloc a unei UAL

Din punctul de vedere al interconectării UAL cu celelalte subansamble, de fapt din punctul de vedere al mecanismului de obţinere a operanzilor şi al furnizării rezultatului există mai multe soluţii posibile :

1°. Utilizarea unei magistrale unice. Schema bloc de interconectare este prezentată în Fig. 1.12

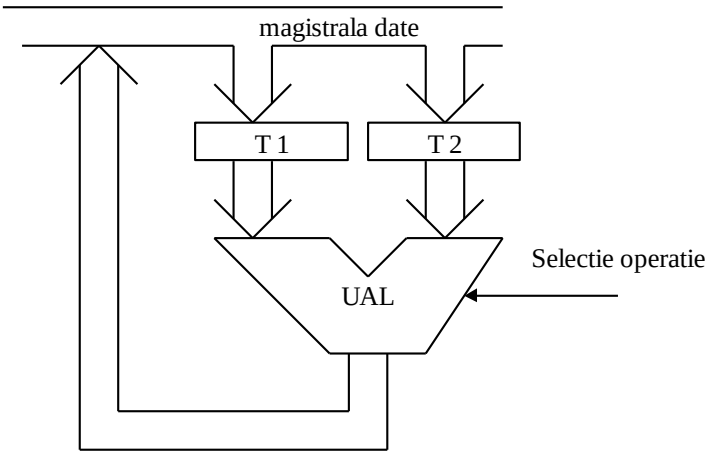


Fig. 1.12 Interconectarea UAL cu o singură magistrală

În acest caz operanzii sunt preluaţi de către UAL din două registre temporare T1 şi T2. Se observă că încărcarea datelor în registrele T1 şi T2 şi transmiterea rezultatului se face prin intermediul unei magistrale locale a UCP. Sursele pentru operanzi şi destinaţia pentru rezultat pot să fie reprezentate de orice registru sau locaţie de memorie. Înainte de efectuarea operaţiei valorile operanzilor trebuie să fie încărcate în registrele temporare T1 şi T2. Se observă că această încărcare nu se poate realiza decât secvenţial.

Dacă sursele celor doi operanzi sunt reprezentate de locaţii de memorie iar rezultatul trebuie să se depună de asemenea în memorie, având în vedere că accesul la memorie nu se poate face decât pentru citirea / scrierea unei singure date la un moment dat, timpul necesar pentru efectuarea operaţiei aritmetice sau logice este dat practic de timpul necesar pentru cele 3 accese la memorie. Dacă operanzii sunt disponibili în registrele UCP atunci din cauza transferului secvenţial al operanzilor în registrele temporare, execuţia instrucţiunii va dura mai mult decât în cazul în care există o legatură directă între registrele care conţin operanzii şi UAL.

2°. Utilizarea unui registru acumulator. Schema bloc de interconectare este prezentată în Fig. 1.13. În acest caz unul dintre opereranzi este preluat întodeauna dintr-un registru special numit registru acumulator. De asemenea rezultatul operaţiei este memorat în registrul acumulator înlocuind vechiul conţinut. Se observă că în acest caz numai sursa unuia dintre operanzi poate să fie constituită de către un registru din UCP sau de către o locaţie de memorie. Deoarece al doilea operand este preluat dintr-un registru UCP sau din registrul de date al memoriei, structura nu mai necesită utilizarea unui registru temporar. Viteza execuţiei instrucţiunilor aritmetice şi logice este mai mare în acest caz deoarece la timpul necesar pentru execuţia efectivă a operaţiei nu se mai adaugă decât timpul necesar pentru pregătirea unui singur operand.

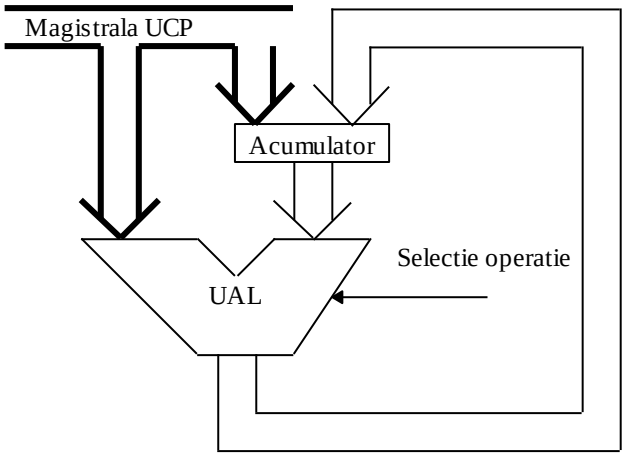


Fig. 1.13. Interconectarea UAL cu registru acumulator

De fapt această structură se poate generaliza şi se pot considera structuri care utilizează mai multe registre ca registre acumulator.

3°. Interconectare UAL cu trei magistrale. Schema bloc de interconectare este prezentată în Fig. 1.14.

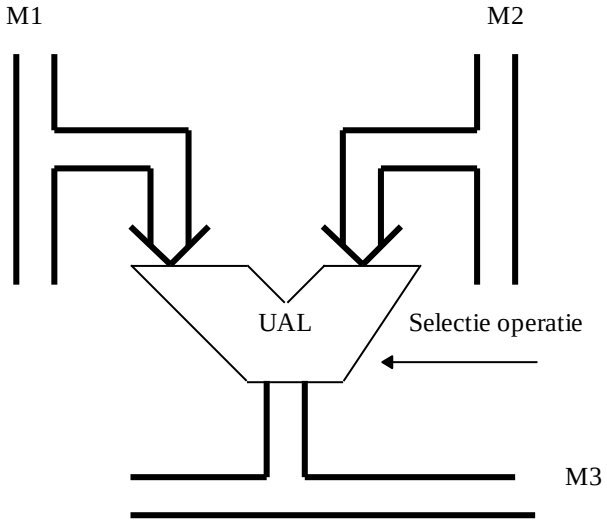


Fig. 1.14 Interconectarea UAL cu 3 magistrale

În acest caz fiecare operand şi rezultatul circulă pe altă magistrală de date. Operanzii sunt preluaţi direct de pe cele două magistrale de date. Rezultatul este transmis pe cea de a 3-a magistrală. Ca şi în prima soluţie atât sursele operanzilor cât şi destinaţia rezultatului operaţiei pot să fie registre ale UCP sau locaţii de memorie. Deoarece obţinerea operanzilor se poate face independent pentru fiecare în parte, în cazul în care nu amândoi operanzii provin din memorie, se observă că cei doi operanzi se pot obţine în paralel. Deci pentru astfel de

situaţii durata execuţiei instrucţiunilor poate să fie mai mică. În cazul în care amândoi operanzii sunt preluaţi din memorie durata aducerii operanzilor este similară cu aceea obţinută pentru prima structură. O variantă a acestei soluţii se obţine dacă se consideră că întodeauna rezultatul va înlocui unul dintre operanzi. În acest caz rezultă o structură cu două magistrale.

1.4.3 Unitatea de comandă

Execuţia unui program rezultă prin realizarea unei secvenţe de aduceri şi execuţii de instrucţiuni din memorie. Adresa de memorie a următoarei instrucţiuni care urmează să se execute este memorată într-un registru special din UCP, registru numit contor program (CP). Când se iniţiază execuţia unui program, în CP se încarcă adresa primei instrucţiuni. În continuare se va executa un ciclu de operaţii care poate să fie descris de organigrama din Fig. 1.15.

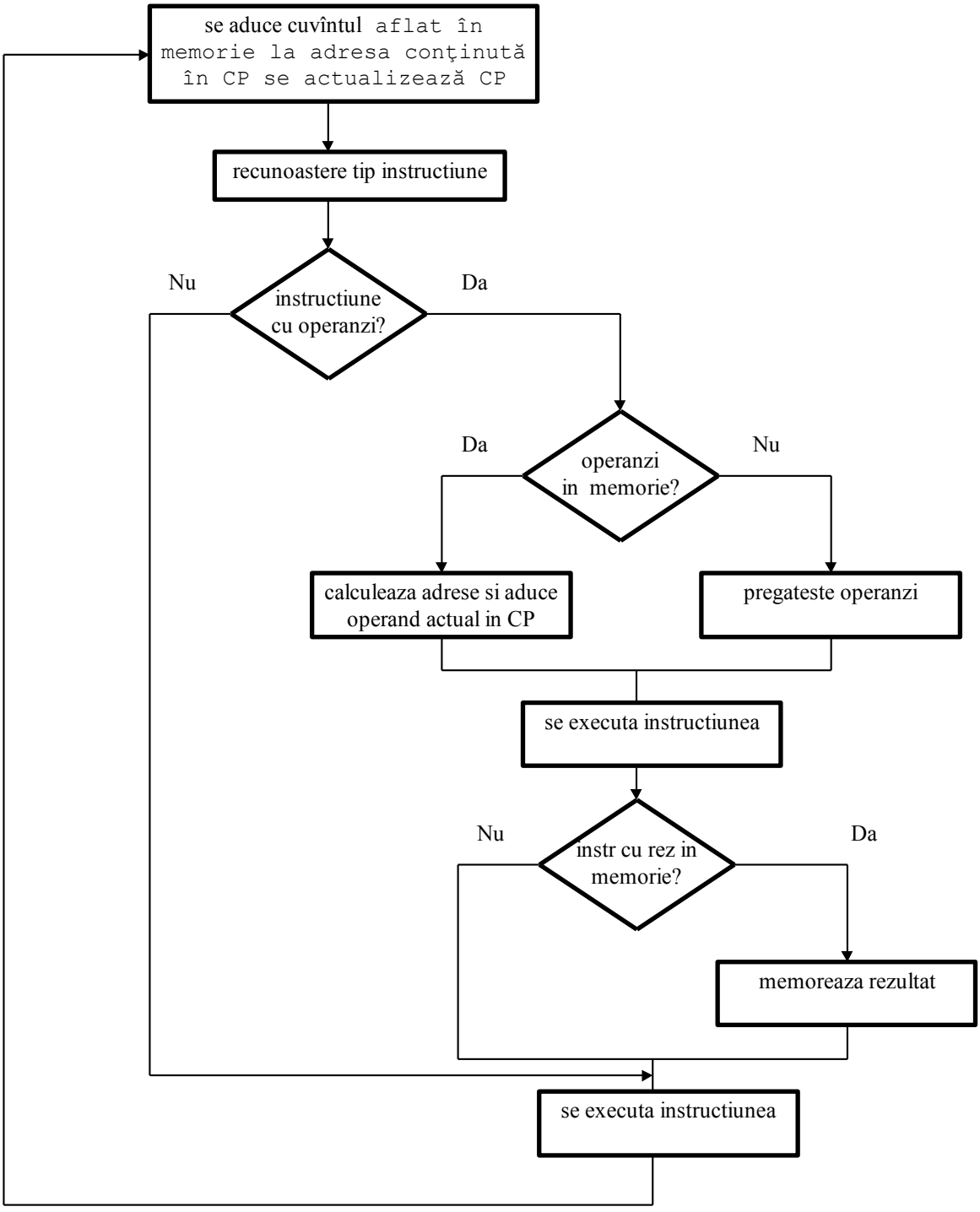


Fig 1.15 Fazele execuţiei unei instrucţiuni

Se observă că în timpul ciclului de aducere şi execuţie a instrucţiunii CP este actualizat astfel încât ori de câte ori se începe ciclul unei noi instrucţiuni conţinutul acestui registru să reprezinte adresa primului cuvânt din instrucţiune. Evident instrucţiunile de control al

execuţiei programului pot să aibă ca efect actualizarea conţinutului registrului CP conform adresei la care se face saltul.

Se observă că din punctul de vedere al interpretării realizate prin execuţia programului de către calculator, un program constă din date şi instrucţiuni, identificarea instrucţiunilor se poate face numai dacă se cunoaşte o adresă de instrucţiune începând de la care se poate face o identificare semnificativă a instrucţiunilor.

Unitatea de comandă (UCd) are rolul de a decodifica şi interpreta instrucţiunile generând semnale de comandă care asigură fluxul de date corespunzător execuţiei instrucţiunii respective. Rezultă deci, că structura UCd este realizată pe baza unităţilor funcţionale pe care aceasta trebuie să le comande şi pe baza setului de instrucţiuni pe care UCP trebuie să-l execute.

În general o unitate de comandă este un automat complex. Există două soluţii de implementare a UCd şi anume sub formă de unităţi de comandă convenţionale şi unităţi de comandă microprogramate. Unităţile de comandă convenţionale sunt realizate de obicei sub forma unei reţele de automate codificate sau complet decodificate.

1.4.4 Setul de instrucţiuni

Orice calculator este controlat de un program, care este format dintr-o secvenţă de instrucţiuni. Fiecare instrucţiune specifică o operaţie care urmează să fie executată de către UCP. În general instrucţiunile unui calculator pot să fie clasificate în câteva clase de bază :

1°. Instrucţiuni pentru transferul datelor. Sursa şi respectiv destinaţia unui astfel de transfer pot să fie constituite de o locaţie de memorie, un registru, sau elemente ale subsistemului de intrare/iesire.

2°. Instrucţiuni aritmetice. Aceste instrucţiuni execută operaţii aritmetice asupra unor date. În acest caz sursa operanzilor şi destinaţia rezultatului pot să fie memoria sau registrele din UCP. Complexitatea operaţiilor care formează setul instrucţiunilor aritmetice diferă mult pentru diferite calculatoare. Astfel, cel mai simplu set posibil de instrucţiuni aritmetice ar putea să conţină numai o instrucţiune de incrementare a conţinutului unui registru sau a conţinutului unui cuvânt din memorie. Pe baza unei astfel de operaţii se poate construi o aritmetică utilizând numere fără semn. În realitate chiar şi cele mai elementare calculatoare "ştiu" să execute operaţia de adunare între două numere (eventual fără semn). La capătul celălalt al scalei complexităţii operaţiilor aritmetice se găsesc calculatoarele în al căror set se găsesc cele patru operaţii aritmetice care se pot efectua asupra numerelor cu sau fără semn reprezentate în virgulă fixă (numere întregi) sau în virgulă mobilă (numere reale).

3°. Instrucţiuni logice şi de deplasare. Aceste instrucţiuni execută operaţii logice (de tip SI, SAU, NU, SAU EXCLUSIV) asupra unor date. Ca şi la instrucţiunile aritmetice sursa operanzilor şi destinaţia rezultatului pot să fie memoria sau registrele din UCP. Toate calculatoarele care conţin în setul lor de instrucţiuni, instrucţiuni logice conţin cel puţin instrucţiuniile SI, SAU şi NU cu toate că după cum se ştie disponibilitatea unei operaţii SI-NU sau SAU-NU ar fi suficientă pentru realizarea oricărei prelucrări de tip logic.

4°. Instrucţiuni de comparaţie. Aceste instrucţiuni pot să fie considerate drept instrucţiuni aritmetice (ca efect al execuţiei unei astfel de instrucţiuni starea UCP este actualizată conform diferenţei dintre cei doi operanzi referiţi în instrucţiune şi acesta este singurul efect al execuţiei instrucţiunii), sau ca instrucţiuni logice (având în vedere semnificaţia operaţiei de comparaţie).

5°. Instrucţiuni pentru controlul execuţiei programelor. Aceste instrucţiuni permit schimbarea dinamică a secvenţei în care se execută instrucţiunile din program producând un salt necondiţionat sau condiţionat la o adresă din program specificată în instrucţiune.

Instrucţiunile sunt memorate în memorie împreună cu datele asupra cărora operează. UCP aduce o instrucţiune, determină ce prelucrări trebuie să se efectueze şi le execută. Fiecare calculator are forma sa proprie de reprezentare a instrucţiunilor, dar orice instrucţiune conţine două informaţii:

- codul operaţiei - care specifică operaţia care urmează să se execute: adunare, comparaţie, salt, etc;
- unul sau mai mulţi specificatori de operanzi care descriu modul în care se obţin operanzii instrucţiunii şi eventual unde se memorează rezultatul acesteia.

În memorie o instrucţiune poate să ocupe unul sau mai multe cuvinte. Există calculatoare la care numărul de cuvinte utilizat pentru reprezentarea instrucţiunilor este fix, în timp ce la alte calculatoare numărul de cuvinte utilizat pentru reprezentarea unei instrucţiuni depinde de tipul acesteia. De exemplu o instrucţiune a unui calculator ipotetic care trebuie să adune conţinutul cuvântului de memorie aflat la adresa 100 cu cel care se găseşte la adresa 200, memorînd rezultatul la adresa 250 ar putea să fie descrisă în memorie cu ajutorul a 4 octeţi dispuşi la adrese consecutive, după cum se arată în Fig.1.16.

ADD
100
200
250

Fig. 1.16 Structura unei instrucţiuni în memorie

În memorie codul operaţiei şi adresele operanzilor ca de altfel şi datele "arată la fel" şi numai modul în care UCP interpretează un număr îl face pe acesta să reprezinte codul unei operaţii, adresa unui operand sau o dată.

1.4.5 Subsistemul de intrare ieşire

Rolul subsistemului de intrare / ieşire este de a conecta echipamentele periferice la calculator. Prin intermediul acestor echipamente periferice se realizează de fapt schimbul de informaţii cu lumea externă care poate să fie reprezentată de operatori umani, alte procesoare, memorii externe, etc.

Cele mai obişnuite echipamente periferice de intrare / ieşire sunt: terminalele de introducere / afişare date, unităţile pentru discuri magnetice, unităţile pentru benzi magnetice, plotere, imprimante, etc.

Se observă că aceste echipamente sunt caracterizate de forme foarte diferite de reprezentare a informaţiei şi de viteze diferite de transfer a informaţiei. De asemenea echipamentele periferice de tip disc sau bandă magnetică pot să transfere la un moment dat blocuri mari de date, în timp ce echipamentele de tip terminal pot să transfere la un moment dat un singur caracter. Corespunzător în subsistemele de intrare / ieşire sunt conţinute interfeţe pentru cuplarea echipamentelor periferice. Rolul acestor interfeţe este de a realiza conversia de format şi de viteză între UCP şi echipamentele periferice.

1.4.6 Limbaje de programare

Un calculator poate să fie utilizat pentru a rezolva orice problemă pentru care se poate defini precis o rezolvare. Prin definire se înţelege în acest caz posibilitatea descompunerii problemei (şi deci a rezolvării) în subprobleme a căror rezolvare presupune execuţia unor comenzi specifice calculatorului. Cu alte cuvinte calculatorul poate să fie utilizat pentru orice problemă pentru care rezolvarea poate să fie descrisă sub forma unui algoritm în termenii operaţiilor specifice calculatorului.

Să considerăm de exemplu problema realizării unui dicţionar român - englez. Problema este foarte simplu de rezolvat, pentru că ceea ce se cere este de fapt căutarea unei perechi de cuvinte într-o listă de echivalenţe. Dificultăţile reale legate de rezolvarea acestei probleme sunt cele referitoare la construirea dicţionarului (formarea bazei de date) şi cele referitoare la regăsirea rapidă a cuvântului căutat pentru cazul în care dicţionarul considerat este mare. Pe de altă parte, problema traducerii din limba română în limba engleză, bineînţeles cu păstrarea semnificaţiei textului este o problemă foarte dificilă datorită înţelesului ascuns al unor combinaţii de cuvinte în funcţie de context. Din acest motiv algoritmizarea acestei probleme

este încă un subiect de cercetare. De altfel, acesta este şi motivul pentru care atât în fazele de specificare a rezolvării unor probleme cu ajutorul calculatorului, cât şi pentru specificarea algoritmilor chiar şi pentru cazuri în care interlocutorul este omul, se preferă utilizarea, în locul limbajului natural, a unor limbaje artificiale pentru care înţelesul fiecărei construcţii este precis (vezi pseudocodul, organigramele, şi alte forme de specificare a algoritmilor). Desigur, ideea creerii unor limbaje artificiale nu este nouă, în sensul că apariţia ei nu este legată de apariţia calculatoarelor. Limbajul matematic, formulele chimice nu reprezintă altceva decât limbaje artificiale create în scopul comunicării în condiţiile anulării oricăror posibilităţi de apariţie a unor formulări ambigue. Apariţia calculatoarelor a dus însă la apariţia unei clase speciale de limbaje artificiale şi anume limbajele de programare. Un limbaj de programare este un limbaj artificial utilizat pentru comunicarea cu calculatorul.

Un limbaj de programare ca de altfel orice limbaj este definit prin: vocabular, sintaxă şi semantică. Vocabularul unui limbaj este format din semnele şi cuvintele din care se pot forma propoziţii. Sintaxa unui limbaj este formată din totalitatea regulilor de bună formare a propoziţiilor, iar semantica unui limbaj este dată de totalitatea regulilor prin care se asociază semnificaţii propoziţilor.

În general un calculator este o maşină care "ştie" să execute un set de operaţii simple numite instrucţiuni. Un program nu este altceva decât o secvenţă de instrucţiuni, efectul execuţiei acestei secvenţe de instrucţiuni fiind o anumită prelucrare de informaţie. Instrucţiuniile "elementare" pe care un calculator poate să le "înţeleagă" şi deci să le execute sunt în general instrucţiuni foarte simple ca de exemplu: adună două numere, compară un număr cu altul, mută o informaţie dintr-o parte a memoriei calculatorului în alta, etc. Setul instrucţiunilor elementare pe care calculatorul îl "înţelege" direct împreună cu regulile de scriere a unor secvenţe de astfel de instrucţiuni formează un limbaj de programare. Acest limbaj este denumit de obicei limbaj maşină, fiind specific maşinii respective.

Există peste o sută de limbaje de programare iar procesul de creare a noi limbaje de programare continuă. În diferite perioade ale istoriei calculatoarelor diferite limbaje de programare au fost considerate ca fiind "cele mai bune". Astfel, dacă la început disputa avea loc între partizanii FORTRAN-ului (1954) şi cei ai COBOL-ului (1959), în prezent disputa are loc între partizanii aceluiaşi FORTRAN (dar mult modernizat devenit FORTRAN 77), şi cei ai limbajelor PASCAL (1971), MODULA (1979), C (1975), ADA (1979), FORTH (1974) PROLOG (1970), LISP (1960). De fapt dacă facem abstracţie de criteriul: " cel mai bun limbaj este cel pe care îl ştiu", criteriul conform căruia tipul problemei determină tipul limbajului în care se descrie rezolvarea problemei, pare cel mai corect. Din punctul de vedere al răspândirii, pînă la apariţia calculatoarelor personale se pare că cele mai multe programe existente în lume erau scrise în COBOL. O dată cu alte mutaţii aduse de apariţia calculatoarelor personale s-a produs o mutaţie şi în domeniul utilizării limbajelor de programare. Astfel, dacă se iau în considerare toate programele scrise în lume, indiferent de

dimensiunea şi performanţele lor, probabil că cele mai multe programe au fost scrise în limbajul BASIC.

O cerinţă fundamentală a oricărui limbaj de programare este asigurarea unei comunicaţii eficiente şi comode între om şi calculator. Problema este însă că ceea ce este comod pentru calculator este foarte nepotrivit pentru utilizator şi invers. Pe de altă parte, dacă la început elementul cel mai "pretenţios" dintre cei doi parteneri era calculatorul şi utilizatorul a trebuit să se adapteze, folosind limbaje de programare foarte apropiate de nivelul operaţiilor pe care le poate executa calculatorul, pe măsură ce progresele tehnologice au permis, au început să se dezvolte limbaje de programare concepute în ideea avantajării utilizatorului. De fapt orice calculator "ştie" să execute un set de operaţii, să le zicem elementare. Dacă descrierea algoritmului de rezolvare a problemei se face chiar în termenii acestor operaţii elementare folosind coduri numerice, se spune că se face programarea în cod masină. Desigur programarea unui calculator la acest nivel este foarte dificilă şi oamenii au scăpat relativ repede de acest nivel. Se consideră de fapt că există două nivele de limbaje de programare: limbaje de programare de nivel scăzut şi limbaje de programare de nivel înalt. Limbajele de programare de nivel scăzut se numesc şi limbaje de asamblare. Fiecare tip de calculator are propriul Său limbaj de asamblare. O operaţie în limbaj de asamblare corespunde unei operaţii în limbaj masină, deosebirea fiind că la nivelul limbajului de asamblare specificarea operaţiilor se face utilizând nume simbolice (ADD, MOV, JMP, JNZ, etc) în timp ce la nivelul limbajului maşină specificarea operaţiilor se face utilizând coduri numerice. Suplimentar faţă de limbajul maşină un limbaj de asamblare dispune şi de alte facilităţi specifice (pseudoinstrucţiuni, macroinstrucţiuni, etc). În cazul limbajelor de nivel superior o operaţie corespunde de obicei cu o secvenţă de operaţii în limbaj maşină, cu alte cuvinte puterea (complexitatea) operaţiilor puse la dispoziţia programatorilor este mult mai mare. Având în vedere observaţia că numărul de linii de program scrise şi verificate pe calculator într-un interval de timp dat de către un programator nu depinde de limbajul de programare utilizat, este evident că cu cât puterea operaţiilor puse la dispoziţie de către limbajul de programare este mai mare, complexitatea problemelor rezolvate cu acelaşi efort de către un acelaşi programator este mai mare. Un alt avantaj al limbajelor de programare de nivel înalt este că în majoritatea cazurilor programele scrise în astfel de limbaje sunt independente de particularităţile calculatoarelor pe care sunt utilizate.

La rândul lor limbajele de programare de nivel înalt pot să fie clasificate după diferite criterii. De exemplu, clasificarea "clasică" în limbaje orientate pentru calcule stiinţifice: FORTRAN, BASIC, APL, etc; limbaje orientate pentru probleme ne numerice: COBOL, LISP, SNOBOL, etc. Limbajele mai noi: C, PASCAL, MODULA, ADA pot să fie utilizate cu bune rezultate pentru ambele tipuri de probleme. Un alt tip de clasificare ţine cont de modul în care se specifică calculatorului ce are de făcut, şi anume există limbaje imperative şi limbaje declarative. Un limbaj imperativ este un limbaj în care programatorul trebuie să descrie pas cu pas prelucrările care trebuie să fie efectuate pentru a rezolva o problemă.

Exemple de limbaje de programare imperative: FORTRAN, PASCAL, COBOL, BASIC, ADA, C, etc.

Aşa cum s-a mai arătat fiecare limbaj de programare a fost inventat pentru a facilita realizarea unui anumit tip de aplicaţii şi nu există şi probabil că nu poate să existe un limbaj de programare bun pentru orice tip de aplicaţii.

Se pune problema însă, cum se realizează implementarea unui limbaj de programare pe un calculator. Există două căi de abordare posibile şi amândouă au fost considerate **importante** la un moment dat. Prima cale este de a realiza calculatoare care să "ştie" limbaje cât mai evoluate şi în istorie au existat numeroase astfel de încercări mai mult sau mai puţin încununate de succes (ca de exemplu calculatorul Burroughs 7400 programabil direct în ALGOL, microprocesorul iAPX 432 programabil în ADA sau mai nou maşinile LISP, MODULA, PROLOG, etc).

A doua cale presupune utilizarea chiar a calculatorului care ştie numai limbajul maşină (îl vom numi LM) pentru a permite programatorului să se exprime într-un limbaj de nivel înalt (pe care îl vom numi LI). Având în vedere că şi LI este destinat efectuării unor prelucrări de date înseamnă că fiecare instrucţiune din acesta poate să fie exprimată cu ajutorul unei secvenţe finite de instrucţiuni din LM. În acest caz procesul de trecere de la o instrucţiune din LI la instrucţiuni din LM poate să fie automatizat, cu alte cuvinte se poate scrie un program care să realizeze traducerea instrucţiunilor din LI în instrucţiuni din LM. Evident acest program va fi scris (cel puţin prima dată) utilizând instrucţiuni din LM. Un astfel de program se numeşte translator. Există două tipuri de translatoare: compilatoare şi interpretoare. Dacă translatorul traduce întreg programul scris în LI în instrucţiuni LM, obţinând un program scris în LM care va putea fi utilizat ca atare ori de câte ori este nevoie, spunem că translatorul este de tip compilator. Dacă translatorul realizează recunoaşterea unei instrucţiuni din LI, execută secvenţa de instrucţiuni LM corespunzătoare şi apoi trece la următoarea instrucţiune din LI, spunem că translatorul este de tip interpretor. Amândouă tipurile de translator permit unui programator în limbajul LI să ignore faptul că calculatorul pentru care scrie programul în LI nu "înţelege" direct acest limbaj. Este ca şi cum la dispoziţia acestui programator se găseşte un alt calculator, un calculator virtual având o arhitectură diferită de a celui real.

Pentru ca operaţia de traducere să fie simplă este bine ca raportul între complexitatea instrucţiunilor disponibile în LI şi în LM să nu fie foarte mare. Corespunzător, limbajul LI poate să fie încă prea simplu pentru descrierea unor aplicaţii semnificative. În acest caz se poate considera un nou limbaj LI' având instrucţiunile mai aproape de situaţia ideală şi se poate utiliza limbajul LI pentru a construi calculatorul virtual care "ştie" limbajul LI', asa cum rezultă din Fig. 1.17.

Desigur trei nu este în acest caz un număr fatidic, deci ierarhia de calculatoare şi deci de limbaje poate să aibă mai multe nivele. Semnificativă este relaţia dintre un limbaj şi un calculator virtual, şi anume orice limbaj defineşte arhitectura unui calculator virtual care "ştie"

să execute programe scrise în acest limbaj. În acelasi timp orice calculator defineşte un limbaj de programare.

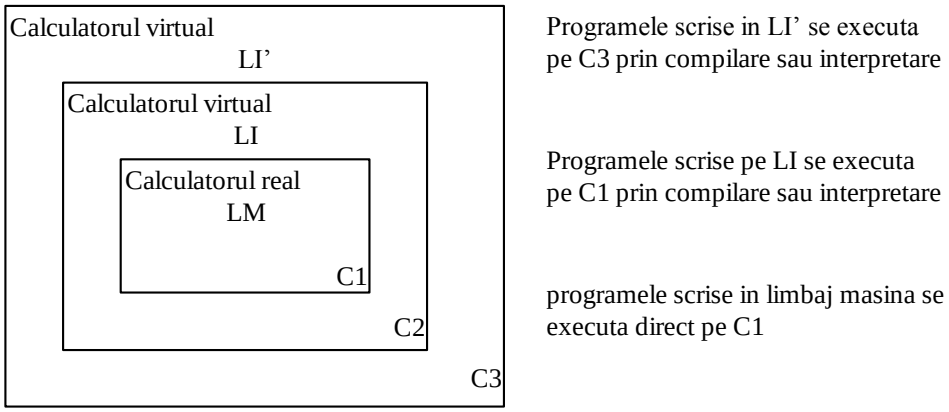


Fig 1.17 Structura ierarhică de limbaje

1.5 Elemente de arhitectură şi structură a unui CN

1.5.1 Concepte de bază

Un sistem de prelucrare numerică a datelor este format dintr-o colecţie de module (unităţi funcţionale) interconectate între ele. Caracteristicile constructive ale acestor unităţi, modul de interconectare a acestora, căile de comunicaţie a datelor formează *structura* sistemului de prelucrare numerică a datelor.

Fiecare modul la rândul lui poate să fie privit ca fiind format dintr-o Unitate de Comandă mai mult sau mai puţin complexă şi resursele care asigură suportul prelucrărilor controlate de unitatea de comandă, reunite sub numele de Unitate de Execuţie. Resursele accesibile programatorului şi modalităţile de control a acestora prin interpretări concrete la nivelul setului de instrucţiuni formează *arhitectura* sistemului de prelucrare numerică a datelor.

Un repertoriu de instrucţiuni este specificat prin :

- formatul instrucţiunilor;
- semantica instrucţiunilor.

Prin *formatul instrucţiunilor* se specifică numărul de cuvinte de memorie utilizat pentru memorarea fiecărui tip de instrucţiune şi semnificaţia câmpurilor care formează instrucţiunea. În general o instrucţiune este compusă din două componente :

- *codul operaţiei* indică operaţia efectuată de către instrucţiune;
- *câmpul de adrese* (prezenţa acestui câmp depinde de tipul operaţiei) specifică modul de obţinere a operanzilor şi eventual de memorare a rezultatului operaţiei.

În proiectarea unui set de instrucţiuni trebuie să se considere o serie de factori ca de exemplu:

- *alegerea lungimii instrucţiunii* influentează performanţele UCP. Dacă viteza de transfer a memoriei este de t cuvinte pe secundă şi numărul mediu de cuvinte utilizate pentru o instrucţiune este r, atunci memoria poate să asigure un flux de maximum t/r instrucţiuni pe secundă. Se observă deci că cu cât instrucţiunile sunt mai scurte cu atât numărul de instrucţiuni executate în unitatea de timp poate să fie mai mare deoarece faza de aducere a unei instrucţiuni durează de regulă mai mult decât timpul de execuţie al unei instrucţiuni;

- *alegerea lungimii câmpului cod operaţie din instrucţiuni* determină numărul de instrucţiuni diferite ce pot fi codificate în cadrul acestui câmp;

- *lungimea cuvintelor de memorie şi* deci *a instrucţiunilor* este de obicei un multiplu al numărului de biţi necesari pentru reprezentarea caracterelor pentru a permite utilizarea eficientă a memoriei pentru memorarea şirurilor de caractere;

- *alegerea lungimii câmpului utilizat pentru specificarea adresei unui operand* determină capacitatea memoriei ce poate fi adresată. Evident unitatea de adresare a memoriei (cuvântul) poate avea la rândul ei diferite lungimi. Dacă de exemplu unitatea de adresare a memoriei este un cuvânt de 32 de biţi atunci capacitatea memoriei adresabile va fi de patru ori mai mare decât în cazul în care unitatea de adresare a memoriei este un cuvânt de 8 biţi, pentru aceasi lungime a adresei.

Să considerăm un format de instrucţiune compusă din codul operaţiei şi o adresă. Să presupunem că primul cîmp ocupă k biţi iar al doilea n biţi. Rezultă deci că este posibilă codificarea a 2^k operaţii diferite iar spaţiul de memorie adresabil este de 2^n adrese diferite. Dacă acelaşi număr n + k de biţi se împarte în k-1 pentru câmpul codului şi n+1 pentru adresă se pot codifica de două ori mai puţine instrucţiuni dar în schimb se va putea adresa o memorie cu o capacitate dublă. Se observă deci că în alegerea dimensiunii câmpurilor trebuie acceptat un compromis între posibilitatea de a codifica cât mai multe tipuri de instrucţiuni şi posibilitatea de a adresa un spaţiu de memorie cât mai mare.

Să considerăm o UCP având instrucţiuni de 16 biţi pentru care adresele se pot specifica pe 4 biţi. O soluţie de proiectare constă din realizarea unor instrucţiuni având codul operaţiei de 4 biţi şi trei câmpuri de adresă pentru doi operanzi şi un rezultat. În acest caz rezultă că sunt posibile numai 16 tipuri de instrucţiuni. O soluţie care permite codificarea unui număr mai mare de instrucţiuni constă din utilizarea mai multor formate de instrucţiuni cu aceaşi lungime, aşa cum se vede în tabelul următor.

Cod operaţie	rezultat	operand1	operand2	observaţtt
0000	xxxx	xxxx	xxxx	15 instrucţiuni cu 3 adrese
0001	xxxx	xxxx	xxxx	
.....				
1110	xxxx	xxxx	xxxx	
Cod operaţie		operand1	operand2	observaţtt
1111	0000	xxxx	xxxx	14 instrucţiuni cu 2 adrese
1111	0001	xxxx	xxxx	
.....				
1111	1101	xxxx	xxxx	
Cod operaţie			operand	observaţtt
1111	1110	0000	xxxx	31 instrucţiuni cu o adresa
1111	1110	0001	xxxx	
.....				
1111	1111	1110	xxxx	
Cod operaţie				observaţii
1111	1111	1111	0000	16 instrucţiuni
1111	1111	1111	0001	
....				
1111	1111	1111	1111	

Se observă că în acest mod cu 16 biţi se pot specifica 76 de instrucţiuni diferite.

O altă soluţie constă din utilizarea unui format de instrucţiuni care să admită instrucţiuni de lungimi diferite. În acest caz se obţine o utilizare mai bună a memoriei în sensul că o instrucţiune pentru care trebuie să se specifice mai puţine informaţii poate să ocupe un spaţiu de memorie mai mic. În acest caz însă structura Unităţii de Comandă este mai complexă.

1.5.2 Moduri de adresare

Instrucţiunile pot să fie clasificate în funcţie de numărul de adrese pe care îl specifică. Numărul de adrese dintr-o instrucţiune depinde de semantica instrucţiunii. Astfel pentru o instrucţiune care realizează de exemplu operaţia de adunare se pot specifica :

- adresele operanzilor şi a rezultatului, rezultând o instrucţiune cu *trei adrese*;
- adresele a doi operanzi dacă rezultatul înlocuieşte unul dintre operanzi, rezultând o instrucţiune cu *două adrese*;
- adresa unui operand dacă celălalt operand şi rezultatul sunt memorate în mod implicit într-un registru de tip acumulator, rezultând o instrucţiune cu *o adresă*;
- nici o adresă dacă adunarea se face între doi operanzi situaţi în vârful stivei, rezultatul fiind plasat în vârful stivei, rezultând astfel o instrucţiune cu *zero adrese*.

Considerând că registrele reprezintă şi ele un spaţiu adresabil se poate considera că o instrucţiune care adună conţinutul a două registre memorând rezultatul în unul dintre acestea este o instrucţiune cu două adrese ca şi instrucţiunile care realizează aceleaşi operaţii asupra unor locaţii de memorie.

Pentru instrucţiunile de salt adresa la care se face saltul reprezintă operandul instrucţiunii. Rezultă corespunzător că o instrucţiune de salt este o instrucţiune cu o adresă.

Ceea ce se memorează într-o instrucţiune pentru un operand sau pentru un rezultat este de fapt informaţia necesară pentru determinarea valorii operandului, respectiv adresa rezultatului. Mecanismul prin care pornind de la informaţia conţinută în instrucţiune se ajunge la valoarea operandului respectiv la adresa rezultatului poartă numele de *mod de adresare*. Numărul mare de moduri de adresare disponibile în calculatoarele moderne a rezultat din necesitatea de a asigura o implementare cât mai simplă şi eficientă a unor structuri de date complexe.

În cele ce urmează utilizăm următoarele notaţii:

r	- registru accesibil programatorilor;
(r)	- conţinutul registrului r;
M[x]	- conţinutul locaţiei de memorie aflată la adresa x;
d	- valoarea unui deplasament conţinut în instrucţiune;

Prezentăm în continuare cele mai uzuale moduri de adresare caracteristice calculatoarelor moderne.

1°. Adresare la registre

În acest caz operandul este conţinut într-un registru specificat în instrucţiune. Obţinerea valorii operandului nu necesită accese la memorie.

2°. Adresare prin registre

Pentru acest mod de adresare instrucţiunea conţine specificarea unui registru r al cărui conţinut reprezintă adresa operandului. Rezultă deci că valoarea operandului este M[(r)]

3°. Adresare prin registre cu incrementare sau decrementare

Funcţionarea acestui mod de adresare este similară cu a modului anterior din punctul de vedere al mecanismului de obţinere a valorii operandului. În mod suplimentar, conţinutul registrului utilizat este actualizat prin incrementare sau decrementare înainte sau după obţinerea valorii operandului. Valoarea cu care se face incrementarea sau decrementarea este egală cu lungimea ocupată în memorie de operand. Deci valoarea operandului este M[(r) + N] sau M[(r) - N] iar conţinutul registrului r devine (r) + N sau (r) - N unde N este lungimea operandului.

În cazul în care acest tip de adresare se utilizează în legătura cu registrul CP se obţine *modul de adresare imediat* pentru care valoarea operandului este conţinută în instrucţiune. Evident actualizarea registrului utilizat (CP) se face după obţinerea valorii operandului.

În cazul în care registrul utilizat este registrul SP acest mod de adresare realizează implementarea unei memorii de tip stivă pentru care accesul este permis prin intermediul registrului SP numai la ultima informaţie memorată. Pentru adresarea în stivă există de obicei două tipuri de adresare: adresare pentru memorare (PUSH). pentru care actualizarea conţinutului registrului SP se face înainte de accesul la memorie şi adresare pentru extragere din stivă (POP) pentru care actualizarea conţinutului registrului SP se face după accesul la memorie.

În general pentru un repertoriu de instrucţiuni se prevede un mod de adresare cu decrementarea registrului înainte de referirea operandului (predecrementare) şi un mod de adresare cu incrementarea registrului după referirea operandului (postincrementare). În acest mod se poate realiza implementarea unei stive care creşte de la adrese mari spre adrese mici. Dacă se alege cealaltă pereche posibilă: incrementare înainte de referirea operandului şi decrementare după referirea operandului se poate implementa o stivă care creşte de la adrese mici spre adrese mari.

4°. Adresare prin registre indirectă cu incrementare sau decrementare

Acest mod de adresare se obţine adăugînd un nivel de indirectare la modul de adresare anterior. În cazul în care operaţia de actualizare a conţinutului registrului r implicat în acest mod de adresare se face înainte de obţinerea operandului atunci valoarea operandului este M [M [(r) + N]] sau M [M [(r) - N]] iar conţinutul registrului r devine (r) + N sau (r) - N unde N este lungimea operandului. În cazul în care operaţia de actualizare a conţinutului registrului r implicat în acest mod de adresare se face după obţinerea operandului atunci valoarea operandului este M [M [(r)]] cu acelaşi tip de actualizare al conţinutului registrului r. În cazul în care registrul utilizat este CP şi incrementarea se face după obţinerea

operandului se obţine *modul de adresare directă*. Pentru acest tip de adresare, adresa operandului este conţinută în instrucţiune. Dacă se mai adaugă un nivel de indirectare, adică adresa conţinută în instrucţiune este adresa la care se găseşte în memorie adresa operandului atunci se obţine *modul de adresare indirectă*.

5°. Adresare bazată

Se numeşte registru de bază un registru al cărui conţinut este utilizat pentru un calcul de adresă. Dacă r este un registru de bază atunci valoarea operandului este M[(r)+d]. Se observă că adresarea prin registru este o adresare bazată pentru care valoarea deplasamentului este zero. Deoarece valoarea deplasamentului este conţinută în instrucţiune, obţinerea sa poate presupune o citire din memorie. În general adresarea bazată este utilizată pentru extinderea spaţiului adresabil. Să considerăm de exemplu un format de instrucţiune pentru care se utilizează k biţi pentru un câmp de adresă. În acest caz spaţiul adresabil are capacitatea de 2^k. Dacă se utilizează însă un registru de bază format din m biţi, se observă că spaţiul de 2^k biţi poate să fie dispus oriunde într-un spaţiu de memorie cu capacitatea de $2^m + 2^k$. De regulă m > k. De exemplu dacă k = 10 şi m = 16 se obţine accesul la zone de memorie de 1K în cadrul unei memorii cu o capacitate de 64K.

În situaţia în care numărul de biţi afectaţi pentru d este suficient pentru a permite accesul la întreaga memorie se obţine *modul de adresare indexată* în acest caz registrul r se numeşte registru index. Modurile de adresare bazată sau indexată pot să fie utilizate pentru implementarea accesului la un vector de date aflate în memorie la adrese succesive. Actualizarea registrului de bază (index) necesară pentru trecerea de la un element al vectorului la altul presupune adunarea (scăderea) la conţinutul curent al registrului a lungimii unui element al vectorului. Se observă că din cele două componente care participă la calculul adresei operandului una este fixă şi anume valoarea d conţinută în instrucţiune. Evident această valoare poate să fie interpretată ca adresa de început a vectorului sau ca indice în vector.

Dacă registrul CP este utilizat ca registru de bază se obţine *modul de adresare autorelativă*. Acest tip de adresare se utilizează în general numai pentru instrucţiunile de salt. Ceea ce se specifică în instrucţiune este distanţa între adresa instrucţiunii curente şi a operandului referit de către instrucţiune. În acest ultim caz deplasarea trebuie să fie un număr întreg cu semn pentru a permite referiri înainte şi înapoi.

6°. Adresare bazată şi indexată

În cazul acestui tip de adresare se utilizează pentru calculul adresei conţinutul a două registre r1 şi r2. Valoarea operandului este în acest caz M [(r1) + (r2)]. Acest tip de adresare este util pentru implementarea accesului la elementele unui vector; pentru care r1 conţine adresa de început a vectorului iar r2 conţine valoarea indexului în acest vector pentru elementul referit, înmulţită cu lungimea unui element. Se observă că în acest caz cei doi

termeni care participă la calculul adresei pot să fie modificaţi, cu alte cuvinte atât adresa de început cât şi indicele elementului referit pot să varieze spre deosebire de cazul adresării indexate.

Pe baza modurilor de adresare considerate anterior se pot construi şi alte moduri de adresare prin adăugarea unor noi nivele de indirectare şi prin adăugarea unor operaţii de actualizare a conţinutului registrelor conţinute în instrucţiune înainte sau după efectuarea referirii la operand.